

The Witch's Dream Night

상세 개발 보고서

보고서 목적

본 문서는 Unity와 Ink 기반으로 개발 중인 2D 비주얼 노벨 프로젝트의 구조, 구현 기능, 문제 해결 과정, 현재 한계 및 향후 개선 방향을 정리한 상세 개발 보고서이다.

항목	내용
프로젝트 유형	2D 비주얼 노벨 게임
개발 엔진	Unity 6000.3.5f2
주요 기술	Ink, UGUI, TextMeshPro, URP 2D, JSON Save
작성일	2026년 5월 1일

1. 프로젝트 개요

본 프로젝트는 Unity 기반의 2D 비주얼 노벨 게임 The Witch's Dream Night 를 개발하는 것을 목표로 한다. 플레이어는 텍스트 기반 스토리를 따라가며 등장인물과 상호작용하고, 선택지와 장면 전환, 배경 및 캐릭터 연출을 통해 이야기의 흐름을 경험한다.

개발 환경은 Unity 6000.3.5f2 이며, 스토리 진행 시스템에는 Ink 를 사용하였다. Ink 를 통해 대사, 화자, 선택지, 장면 연출 태그를 작성하고, Unity 내부에서는 이를 해석하여 UI 와 연출 시스템에 반영하는 구조로 설계하였다.

본 프로젝트의 핵심 목표는 단순히 대사를 출력하는 시스템을 만드는 것에 그치지 않고, 실제 비주얼 노벨 게임에서 필요한 저장/불러오기, 백로그, 오토 모드, 스킵 모드, 옵션 설정, 타이틀 화면, 배경 및 캐릭터 연출 등을 모듈화된 구조로 구현하는 것이다.

2. 개발 환경 및 사용 기술

구분	사용 기술	역할
게임 엔진	Unity 6000.3.5f2	2D 비주얼 노벨 실행 환경
렌더링	Universal Render Pipeline 2D	2D 화면 및 조명 기반 구성
스토리	Ink / ink-unity-integration	대사, 선택지, 태그 기반 연출 스크립트
UI	UGUI, TextMeshPro	대사창, 버튼, 옵션, 저장 슬롯 표현
저장	JSON Local Save	슬롯 기반 저장 및 복원
버전 관리	Git	개발 이력 및 변경 사항 관리

Ink 는 비주얼 노벨의 스토리 흐름을 텍스트 기반으로 작성하기에 적합하며, Unity 와 연동하여 대사 출력, 선택지, 태그 기반 연출을 구현할 수 있다는 장점이 있다. 본 프로젝트에서는 Ink 의 태그 기능을 활용하여 #speaker, #bg, #ch, #bgm, #shake 등의 명령을 작성하고, Unity 코드에서 이를 해석하여 화면 연출로 연결하였다.

3. 프로젝트 구조

- Assets/Scripts: 게임의 주요 C# 스크립트

- Assets/Scripts/Editor: Unity 에디터 자동화 도구
- Assets/Ink: Ink 원본 스토리와 컴파일된 JSON 파일
- Assets/Scenes: 게임 씬
- Assets/ScriptObjects: 비주얼 노벨용 에셋 데이터베이스
- Assets/Prefabs: 선택지 버튼, 백로그 항목 등 UI 프리팹
- ProjectSettings, Packages: Unity 프로젝트 설정 및 패키지 정보

현재 주요 씬은 SampleScene 과 TitleScene 으로 나뉜다. SampleScene 은 실제 게임 진행 화면이며, 대사 출력, 배경 전환, 캐릭터 표시, 백로그, 저장/불러오기, 옵션 UI 가 포함되어 있다. TitleScene 은 타이틀 화면으로, 새 게임, 이어하기, 불러오기, 옵션, 종료 버튼을 제공한다.

4. 전체 시스템 아키텍처

계층	주요 클래스	책임
Story	InkStoryEngine	Ink JSON 로드, 대사 및 선택지 반환, Ink 상태 저장/복원
Flow Control	VNDirector	스토리 루프, 태그 처리 요청, UI 출력, 세이브/로드 흐름 제어
UI	VNUIController 외 하위 모듈	대사창, 선택지, 백로그, 옵션, 저장/로드 패널 관리
Presentation	VNPresenter, VNCommandProcessor	태그 해석, 배경/캐릭터/BGM/SFX/화면 흔들림 연출
Data / Save	VNSaveData, VNSaveService	JSON 기반 슬롯 저장 및 로컬 복원
Title	VNTitleManager	타이틀 화면, 새 게임, 이어하기, 옵션, 로드 분기

4.1 Story Layer

InkStoryEngine 은 Ink JSON 파일을 로드하고, 현재 스토리 상태에서 다음 대사나 선택지를 반환하는 역할을 담당한다. Ink 런타임의 Story 객체를 감싸는 형태로 구현되어 있어 Unity 의 다른 시스템이 Ink 내부 구조에 직접 의존하지 않도록 한다.

- Ink JSON 초기화

- 다음 대사 및 현재 선택지 반환
- 선택지 선택 처리
- 특정 Knot 으로 이동
- Ink 상태 JSON 저장 및 복원
- #speaker 태그에서 화자 정보 추출

4.2 Flow Control Layer

VNDirector 는 게임 진행의 중심 역할을 담당한다. InkStoryEngine 에서 대사와 태그를 받아오고, 태그는 연출 처리기로 전달하며, 대사는 UI 에 출력한다. 저장과 불러오기에서도 Ink 상태, 배경, 캐릭터, BGM, 백로그를 함께 관리한다.

4.3 UI Layer

VNUIController 는 게임 화면의 UI 흐름을 담당한다. 실제 세부 기능은 VNTextTyper, VNChoicePanel, VNBacklogManager, VNOptionPanel, VNSaveLoadPanel, VNSaveSlotItem 으로 분리되어 있다. 이러한 분리는 UI 기능을 수정할 때 영향 범위를 줄이고, 각 컴포넌트의 책임을 명확하게 만든다.

4.4 Presentation Layer

VNPresenter 는 화면 연출을 담당한다. Ink 태그에서 전달된 배경, 캐릭터, 사운드, 흔들림 명령을 실제 Unity 화면에 반영한다. VNCommandProcessor 는 Ink 태그 문자열을 해석하여 VNPresenter 가 이해할 수 있는 호출로 변환한다.

4.5 Save Layer

저장 시스템은 단순히 마지막 대사만 저장하지 않고, Ink 상태 전체와 화면 상태를 함께 저장한다. VNSaveData 에는 저장 시각, Ink 상태 JSON, 챕터명, 플레이 시간, 마지막 화자와 대사, 백로그, 현재 배경, 현재 BGM, 활성 캐릭터 목록이 포함된다.

5. 주요 구현 기능

기능	구현 내용
Ink 기반 진행	main.ink 와 main.json 을 기반으로 대사, 선택지, 태그를 처리한다.
대사 출력	VNTextTyper 를 통해 타이핑 효과와 클릭 스킵을 제공한다.
선택지	Ink 선택지를 버튼으로 동적 생성하고 선택 결과를 Ink 엔진에 전달한다.
백로그	지나간 대사를 최대 100 개까지 보관하고 세이브 데이터에도 포함한다.
저장/불러오기	슬롯 기반 JSON 저장으로 Ink 상태와 화면 상태를 함께 복원한다.
타이틀 화면	새 게임, 이어하기, 불러오기, 옵션, 종료 흐름을 제공한다.
옵션	볼륨, 텍스트 속도, 오토 대기 시간, 해상도, 전체 화면 설정을 관리한다.
오토/스킵	대사 길이 기반 자동 진행과 고속 진행 모드를 지원한다.
연출	배경 전환, 캐릭터 표시, 페이드, 화면 흔들림, BGM/SFX 를 처리한다.

6. 에디터 자동화 도구

프로젝트에는 Unity 에디터에서 UI 구조를 자동 생성하거나 연결하는 도구가 포함되어 있다. 이 도구들은 반복적인 수동 배치 작업을 줄이고, UI 참조 누락으로 발생하는 오류를 줄이기 위해 작성되었다.

도구	역할
VNCoreSetupTool	핵심 시스템 오브젝트 생성 및 연결
VNUISetupTool	UI 모듈, 옵션 UI, 타이틀 UI 생성
VNSaveLoadUIBuilder	저장/불러오기 UI 생성
VNBacklogSetupTool	백로그 UI 생성

7. 주요 문제 해결 과정

문제	해결 내용
백로그 UI 구조 개선	단순 텍스트 기반 구조를 VNBacklogManager 와 VNBacklogEntry 로 분리하여 확장성과 복원성을 높였다.
오디오 연결 구조 개선	VNPresenter 와 VNTitleManager 가 AudioSource 를 런타임에 자동 생성하도록 하여 참조 누락 문제를 완화했다.
타이틀 옵션창 닫기 문제	타이틀 모드에서는 menuRoot 가 없을 수 있으므로 Close()로 닫히도록 분기 처리했다.
세이브 슬롯 정보 표시	날짜, 플레이 시간, 챕터명, 배경 썸네일 복원을 위한 런타임 보정과 레이아웃 정리를 추가했다.
로드 후 화면 흔들림 지속	프레젠티어의 일시 연출 상태를 ResetTransientState()로 초기화하고 흔들기 루틴을 unscaledDeltaTime 기반으로 변경했다.
저장 후 배경 전환 멈춤	저장/로드 패널 닫기 시 timeScale 을 복구하고, 배경/캐릭터 페이드를 unscaledDeltaTime 기반으로 변경했다.

최근 버그 수정 요약

배경 전환 직전 저장 후 다음 대사로 진행하면 #bg ... fade 전환이 끝나지 않는 문제가 있었다. 원인은 옵션 메뉴에서 저장 패널을 열 때 Time.timeScale = 0 상태가 유지되고, 배경 페이드가 Time.deltaTime 에 의존했기 때문이다. VNSaveLoadPanel.Close()에서 timeScale 을 복구하고, VNPresenter 의 페이드 루틴을 unscaledDeltaTime 기반으로 바꾸어 해결했다.

8. 시행착오와 의사결정

개발 과정에서는 기능을 단순히 추가하는 것보다, 각 기능이 기존 시스템과 어떻게 연결되는지를 계속 확인하는 일이 중요했다. 특히 비주얼 노벨 시스템은 대사 진행, UI, 연출, 저장 상태가 서로 긴밀하게 연결되어 있어 한 부분의 수정이 다른 부분에 예상치 못한 영향을 줄 수 있었다.

시행착오	문제 상황	결정 및 개선
------	-------	---------

단일 UI 컨트롤러 비대화	초기에는 대사, 백로그, 선택지, 저장 UI 가 한 흐름에 몰리면서 수정 범위가 커졌다.	타이핑, 선택지, 백로그, 저장 패널을 별도 컴포넌트로 분리하여 역할을 명확히 했다.
수동 참조 연결 의존	Unity 씬에서 Inspector 참조가 비어 있으면 기능이 조용히 동작하지 않는 문제가 있었다.	런타임 자동 탐색 및 에디터 자동화 도구를 추가하여 참조 누락에 더 강한 구조로 바꾸었다.
저장 데이터 범위 판단	마지막 대사만 저장하면 배경, 캐릭터, BGM 이 복원되지 않아 플레이 경험이 어색해졌다.	Ink 상태와 함께 배경, 캐릭터, BGM, 백로그를 저장하는 방식으로 확장했다.
timeScale 처리	옵션 메뉴와 저장/로드 패널에서 일시정지 상태가 남아 연출 코루틴이 멈추는 문제가 발생했다.	중요 연출은 unscaledDeltaTime 기반으로 통일하고, 패널 종료 시 timeScale 복구를 보장했다.

이 과정에서 가장 크게 느낀 점은 "기능 구현"과 "상태 복원"은 별개의 문제라는 것이다. 화면에 한 번 정상적으로 표시되는 기능이라도, 저장과 로드, 옵션 메뉴, 타이틀 씬 전환 같은 흐름을 거치면 완전히 다른 조건에서 다시 동작해야 한다. 따라서 구현 후에는 단순 실행뿐 아니라 씬 전환, 로드, 일시정지, 스킵 모드 같은 주변 조건과 함께 검증해야 했다.

9. 개발 과정에서 배운 점

- 비주얼 노벨 구조에서는 스토리 데이터와 화면 연출을 분리해야 유지보수가 쉬워진다.
- Ink 태그를 활용하면 시나리오 파일에서 연출을 직접 지정할 수 있어 작업 흐름이 유연해진다.
- Unity 의 Time.timeScale 은 UI 와 연출 코루틴에 큰 영향을 주므로, 일시정지 중에도 동작해야 하는 기능은 unscaledDeltaTime 을 사용해야 한다.
- 저장/불러오기는 단순 데이터 저장이 아니라 현재 화면 상태를 다시 구성하는 과정이므로, 복원 순서가 중요하다.
- Inspector 참조 누락은 초기에 발견하기 어렵기 때문에, 자동 연결 또는 자가 복구 로직이 안정성에 도움이 된다.
- 기능을 한 클래스에 계속 추가하기보다 책임 단위로 분리하면 이후 버그 수정과 기능 확장이 쉬워진다.

가장 중요한 학습 포인트

이번 프로젝트에서 가장 중요한 배움은 UI, 스토리, 연출, 저장 시스템을 각각 따로 생각하면 안 된다는 점이었다. 실제 플레이에서는 이 네 가지가 동시에 움직이기 때문에, 한 기능을 만들 때도 저장 후 복원, 일시정지 후 재개, 타이틀에서 진입하는 경우까지 함께 고려해야 했다.

10. 개발 중 고민했던 점

개발 중 가장 많이 고민한 부분은 시스템을 어디까지 자동화하고, 어디까지 수동으로 Unity Inspector 에서 관리할 것인가였다. 수동 연결은 빠르게 구현할 수 있지만, 씬이 늘어나거나 UI 를 다시 생성할 때 참조가 비는 문제가 반복될 수 있다. 반대로 자동화 코드를 많이 넣으면 초기 구현량은 늘어나지만, 이후 유지보수와 재생성에는 유리하다.

또 다른 고민은 저장 데이터의 범위였다. 저장 파일에 너무 적은 정보만 담으면 복원이 부정확하고, 너무 많은 정보를 담으면 구조가 복잡해진다. 본 프로젝트에서는 플레이어가 저장한 장면을 자연스럽게 이어갈 수 있도록 Ink 상태, 마지막 대사, 배경, 캐릭터, BGM, 백로그를 함께 저장하는 방향을 선택하였다.

마지막으로 고민한 부분은 기존 구조를 유지하면서 버그를 해결하는 방식이었다. 예를 들어 오디오 시스템을 별도의 전역 매니저로 새로 만들 수도 있었지만, 현재 단계에서는 VNPresenter 와 VNTitleManager 의 기존 책임 안에서 자동 AudioSource 생성과 볼륨 적용을 보강하는 쪽이 프로젝트 규모에 더 적합하다고 판단하였다.

11. 현재 프로젝트의 장점

- 스토리, 흐름 제어, UI, 연출, 저장 시스템의 책임이 비교적 명확하게 분리되어 있다.
- Ink 태그를 통해 코드 수정 없이 스토리 파일에서 장면 연출을 지정할 수 있다.
- 저장/불러오기 시스템이 대사뿐 아니라 배경, 캐릭터, BGM, 백로그까지 함께 복원한다.
- 에디터 자동화 도구를 통해 복잡한 UI 구조를 일관되게 생성할 수 있다.
- 로드 및 저장 이후 발생할 수 있는 시간 정지, 코루틴 잔존 문제를 점진적으로 방어하고 있다.

12. 현재 한계 및 개선 필요 사항

개선 항목	내용
-------	----

오디오 에셋 등록	오디오 재생 구조는 준비되어 있으나 VNAssetDatabase 에 실제 BGM/SFX 키와 AudioClip 등록이 필요하다.
빌드 시작 씬	현재 빌드 순서가 SampleScene 에서 시작할 수 있으므로 타이틀 시작 의도라면 TitleScene 우선 배치를 검토해야 한다.
TMP 경고 정리	TMP_Text.enableWordWrapping obsolete 경고를 최신 textWrappingMode API 로 변경할 필요가 있다.
캐릭터 연출 확장	캐릭터 흔들림, 이동 애니메이션, 표정 전환 효과 등 추가 연출을 확장할 수 있다.
장르 부가 기능	CG 갤러리, 회상 모드, 수집 요소 등 비주얼 노벨 특화 기능을 추가할 수 있다.

13. 향후 개발 계획

- 실제 BGM/SFX 에셋 등록 및 키 정책 정리
- 빌드 시작 씬을 TitleScene 으로 변경할지 검토
- TextMeshPro obsolete 경고 정리
- 캐릭터 연출 기능 확장
- 저장 슬롯 썸네일 표시 품질 개선
- CG 갤러리 및 수집 요소 추가
- 시나리오 분량 확장 및 선택지 분기 강화
- 전체 UI 디자인 통일 및 폴리싱

14. 결론

The Witch's Dream Night 는 Unity 와 Ink 를 기반으로 한 2D 비주얼 노벨 프로젝트로, 대사 진행, 선택지, 배경 및 캐릭터 연출, 저장/불러오기, 백로그, 옵션, 타이틀 화면 등 장르에 필요한 주요 기능을 구현하였다.

개발 과정에서 단순 기능 추가뿐 아니라, UI 와 연출, 저장 시스템 간의 충돌을 해결하며 구조를 점진적으로 개선하였다. 특히 로드 후 연출 상태 초기화, 저장 패널 이후 시간 복구, 배경 페이드 멈춤 문제 해결 등은 실제 플레이 흐름에서 발생할 수 있는 버그를 분석하고 안정성을 높인 사례이다.

현재 프로젝트는 핵심 시스템의 기반이 어느 정도 갖추어진 상태이며, 향후 실제 사운드 에셋 등록, UI 폴리싱, 캐릭터 연출 확장, 시나리오 분기 추가를 통해 완성도를 높일 수 있다.